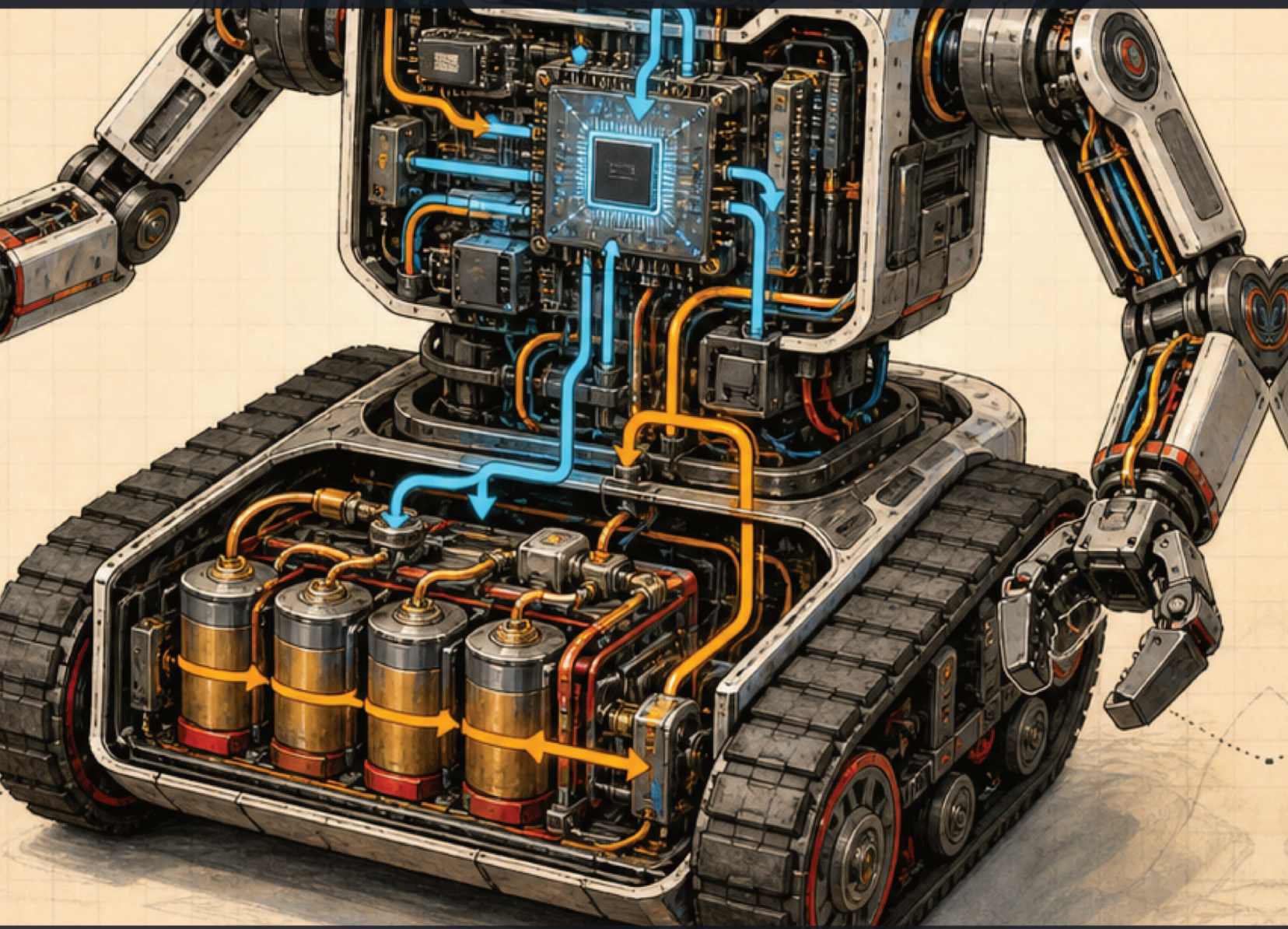
BUILDING R.O.B. - VOLUME 7

ENGINEERING ROBControllerVision

Swift 6, control ownership, dual arms, three camera feeds, and immersive 360



AN IMPLEMENTATION-LEVEL GUIDE FOR SWIFT DEVELOPERS

Rodolfo Aramayo / OrbitusRobotics LLC - First Maker Faire Edition

Write the controller as a safety-critical distributed client

A beautiful spatial interface still needs strict state, time, identity, and ownership boundaries.

This volume reads the current ROBControllerVision source as a serious Swift codebase. It explains why types live in the application or package, how actor isolation and structured concurrency protect lifecycle state, how explicit controller ownership and leased input stop stale motion, how two AMBER arms receive independent dead-man-gated joint authorities, how three authenticated H.264 feeds reach flat and immersive presentation, and how explicit microphone capture becomes bounded text rather than raw audio streaming.

Paths are relative to the public `https://github.com/RudyAramayo/ROBControllerVision` repository, with robot-side counterparts in `https://github.com/RudyAramayo/Cerebro`. Every chapter names the exact source files being analyzed. Code changes after this edition must be checked against the repository, tests, SDK documentation, and physical robot behavior.

First evidence-based draft: 2 August 2026. This historical engineering edition distinguishes documented facts, builder recollection, experiments, plans, and facts not established by the available evidence.

A VISION PRO APP IS NOT THE ROBOT'S FINAL SAFETY LAYER

Client-side arming, dead-man evaluation, leases, and emergency-stop commands are valuable but cannot stop hardware after every possible network, process, power, or actuator failure. Cerebro and lower layers must reject stale input and stop independently. A verified physical emergency stop and controlled test procedure remain mandatory.

The Building R.O.B. library

VOLUME 1	*Meet ROB: A Robot Is a Team of Systems* - ages 8-12
VOLUME 2	*Circuits and Signals with ROB* - ages 10-14
VOLUME 3	*Motion Workshop: Treads, Gears, and Fabrication* - ages 10-15
VOLUME 4	*Mission Control: Arduino, Mac, Networks, and Vision* - ages 12-16
VOLUME 5	*AI, Robotics, and Codex: Directing, Verifying, and Owning AI-Built Systems* - advanced makers
VOLUME 6	*Dual-Arm Robotics: AMBER B1, URDF, CAN, and Ubuntu* - advanced makers
VOLUME 7	*Engineering ROBControllerVision: Swift, Spatial Input, Video, and Speech* - Swift developers
VOLUME 8	*Engineering Cerebro: Perception, Models, H.264, and Robot Coordination* - software engineers
MANUAL	*Building R.O.B.: The Complete Maker's Field Manual* - advanced builders

HOW TO USE THIS BOOK

This is a code-reading and implementation book. Keep the repository open beside the PDF, run package tests after each domain change, compile the visionOS target, and use the simulator before attempting a two-device or powered-hardware test.

HOW TO FIND THE FILES

Open the public repositories on GitHub and follow each printed repository-relative path: `https://github.com/RudyAramayo/ROBBooks`, `https://github.com/RudyAramayo/ROBArduino`, `https://github.com/RudyAramayo/Cerebro`, `https://github.com/RudyAramayo/ROBController`, `https://github.com/RudyAramayo/ROBControllerVision`, `https://github.com/RudyAramayo/Amber-HomeFolder`, and `https://github.com/RudyAramayo/ROBTrainingGames`. The website is published separately at `https://github.com/OrbitusRoboticsWebSite/ORobotics`. See `https://github.com/RudyAramayo/ROBBooks/blob/main/ROB-Books/OPEN-SOURCE-CODE-MAP.md` for clickable repository and file links, license cautions, and renamed-file search instructions. A named archival item without a verified public repository is labeled as evidence, not given an invented URL.

Contents

CHAPTER 1

Read this book with the repository open

ROBControllerVision is a native SwiftUI visionOS controller with a reusable Swift package. It connects to the real Cerebro application through authenticated Network.framework transports or to a deterministic simulator. Its current surface combines explicit drive ownership, independent dual-arm authority, three live camera feeds, and an immersive Insta360 view. The public source is `https://github.com/RudyAramayo/ROBControllerVision`. Clone it, then use this repository root:

```
ROBControllerVision/
```

The application is under `ROBControllerVision/ROBControllerVision`. Reusable domain, video, and Cerebro adapter code is under `ROBControllerVision/Packages/ROBControlCore`. Tests live beside that package. Matching robot-side behavior is in the separate `Cerebro` repository.

> **SOURCE TRAIL — ANALYZING NOW:** `ROBControllerVision/README.md`, `docs/architecture.md`, `docs/protocol-v2.md`, and `docs/real-video-integration.md` describe the intended system. Source and tests decide what the current build actually does.

This book uses **observed** for behavior directly supported by the inspected source, **design requirement** for an invariant the code intends to preserve, and **proposed** for an extension not yet implemented.

The August 23 inspection used local commit `63b9d9e`. That branch was clean but three commits ahead of `origin/main`; the latest panorama-heading/orientation behavior is therefore local source evidence, not yet reproducible from the public remote until those commits are pushed.

CHAPTER 2

Start with the build graph

> **SOURCE TRAIL — ANALYZING NOW:** the Xcode project's `project.pbxproj` and the local package's `Package.swift`. The source map gives their full paths.

The Xcode application target owns visionOS lifecycle, SwiftUI presentation, ARKit, GameController, speech permission, and the app-facing view model. The Swift package declares three libraries:

- **ROBControlCore** contains pure domain types, session state, leases, dead-man logic, simulator, and video-domain contracts.
- **ROBCerebroTransport** contains Network.framework, Security.framework, pairing, discovery, pinned TLS, legacy control compatibility, and the separate video client.
- **ROBVideoPipeline** contains bounded media channels, H.264 support, validation, sample-buffer construction, and synthetic video.

This dependency direction is deliberate. Domain code must not import SwiftUI, GameController, ARKit, or a concrete network adapter. The app may import all three products. The transport imports the domain contract. The video pipeline imports the video-domain types.

The package requires Swift tools 6.0 and supports iOS 18, macOS 15, and visionOS 2. Swift 6 concurrency checking therefore belongs to the architecture, not merely compiler polish.

Reproduce the build

```
swift test --package-path Packages/ROBControlCore

xcodebuild \
  -project ROBControllerVision.xcodeproj \
  -scheme ROBControllerVision \
  -configuration Debug \
  -destination 'generic/platform=visionOS Simulator' \
  -derivedDataPath /tmp/ROBControllerVisionDerivedData \
  CODE_SIGNING_ALLOWED=NO build
```

Change the placeholder bundle identifier and select a development team only when preparing a physical-device build. Do not commit a personal signing identity.

CHAPTER 3

Compose the application at one obvious boundary

SOURCE TRAIL – ANALYZING NOW: `App/ROBControllerVisionApp.swift`, `App/ContentView.swift`, and `App/RobotViewModel.swift`.

The app entry creates one long-lived application model shared by a main cockpit `WindowGroup`, a separate Insta360 window, and a mixed `ImmersiveSpace`. `ContentView` receives the bindable model and composes three cockpit regions in an `HStack`: controls on the left, camera surfaces in the center, and connection, speech, telemetry, and safety information on the right.

The main layout intentionally has no vertical `ScrollView`. The window is 1760 by 920 points; the center camera region is 920 points wide and each side wing is 360 by 820. Side panels receive an eight-degree Y-axis `rotation3DEffect`, a 42-point Z offset, perspective 0.22, and a shadow. This produces depth while keeping the fastest control and safety surfaces in ordinary windows. The panoramic sphere is a separate mixed immersive presentation, not a second control owner.

The central SwiftUI pattern is:

```
struct ContentView: View {
    @Bindable var model: RobotViewModel
    @FocusState private var receivesControllerEvents: Bool

    var body: some View {
        HStack { /* left controls, video, right diagnostics */ }
            .handlesGameControllerEvents(matching: .gamepad)
            .focusable()
            .focused($receivesControllerEvents)
            .task {
                receivesControllerEvents = true
                model.start()
            }
            // App/scene lifetime is coordinated above this view so an
            // immersive transition does not destroy the live media session.
    }
}
```

`@Bindable` projects bindings from an Observation model. `@FocusState` is operational: GameController events depend on the view receiving focus. The `.task` modifier couples asynchronous startup to view presence, while `onDisappear` forces teardown.

Do not hide process ownership in many views. `RobotViewModel` is the application coordinator; feature views render state and invoke named intents.

CHAPTER 4

Use Observation without turning everything into UI state

SOURCE TRAIL — ANALYZING NOW: `App/RobotViewModel.swift`.

`RobotViewModel` is `@MainActor`, `@Observable`, and `final`. That combination communicates three decisions:

1. UI-observable mutation occurs on the main actor.
2. SwiftUI can track properties without `ObservableObject` and `@Published` boilerplate.
3. The class is not designed for inheritance.

Properties used for rendering—snapshot, status messages, selected endpoint, speed, text draft—remain observable. Internal tasks, the session actor, simulator, pairing store, lifecycle IDs, and cached input samples use `@ObservationIgnored`. This prevents observation tracking from becoming an accidental synchronization mechanism.

The initializer performs dependency injection:

```
init(
    session: RobotSession = RobotSession(),
    simulator: SimulatedRobotEndpoint? = nil,
    videoPipeline: VideoPipelineCoordinator = VideoPipelineCoordinator(),
    pairingStore: ROBCerebroPairingStore = ROBCerebroPairingStore()
)
```

Defaults keep production composition easy. Parameters let tests substitute controlled dependencies. The simulator default receives a `SyntheticVideoDataSource`, so offline UI development exercises the real video-domain contract.

Closure callbacks capture `[weak self]`. That avoids a cycle between the coordinator and long-lived input helpers. Callback bodies mutate main-actor state because the helpers themselves are main-actor isolated.

CHAPTER 5

Make lifecycle cancellation explicit

SOURCE TRAIL — ANALYZING NOW: `RobotViewModel.start()`, `stop()`, `setSceneActive(_:)`, and `deinit`.

The controller owns several independent tasks: session updates, connect/disconnect actions, virtual input refresh, video actions, and video-pipeline lifecycle. Each task has a property and an identity when stale completion could race a replacement.

`start()` is idempotent: it returns if the updates task already exists. It starts physical input providers, obtains the session's asynchronous update stream, and iterates with cancellation checks. `stop()` cancels every owned task, invalidates motion, stops video and speech, stops hardware input, and asynchronously disconnects the session.

The scene phase is a safety input. An ordinary inactive transition brakes motion but may preserve negotiated video while the mixed immersive scene becomes active. Backgrounding performs the stronger teardown. Returning active does not silently arm motion. This split matters because visionOS can mark the presenting window inactive while the immersive space is still in use; window disappearance must not be mistaken for process termination.

Use a lifecycle generation or action UUID when cancellation alone is insufficient. A cancelled operation may still unwind later. The completion method checks identity before clearing the current task slot:

```
private func finishAction(_ id: UUID) {
    guard actionID == id else { return }
    actionID = nil
    actionTask = nil
}
```

This is a compact defense against an old task erasing ownership of a newer one.

CHAPTER 6

Put the state machine in an actor

SOURCE TRAIL — ANALYZING NOW: `RobotSession.swift` and `ConnectionModels.swift` in the package's `Connection` directory.

`RobotSession` owns connection phase, handshake, command sequencing, safety evaluation, pending video continuations, timeouts, open media-channel ownership, and the asynchronous snapshot stream. An actor serializes these mutations without forcing network and media work onto the main actor.

The view model never edits a connection snapshot directly. It asks the actor to connect, disconnect, arm, stop, update input, or subscribe. The actor publishes immutable snapshots. This is unidirectional data flow:

```
view intent -> RobotViewModel -> RobotSession actor -> RobotTransport
                                        |
                                        v
                                AsyncStream snapshots
                                        |
                                        v
                                main-actor presentation
```

Pending subscription calls use checked continuations, indexed by subscription ID, with independent timeout tasks. Cancellation removes abandoned IDs and prevents a late reply from resurrecting an abandoned operation. Open data channels remember which transport owns them so teardown calls the correct adapter.

When adding a new request/reply operation, copy this discipline: unique request ID, exactly one continuation, deadline, cancellation handler, late-reply rejection, disconnect cleanup, and tests for every ordering.

CHAPTER 7

Define a domain protocol before a wire format

SOURCE TRAIL – ANALYZING NOW: `Control/ControlProtocol.swift`, `Video/VideoProtocol.swift`, `Video/VideoDataTransport.swift`, and `CerebroRobotTransport.swift`.

`RobotTransport` is a `Sendable` protocol with connect, disconnect, send, and events operations. Application and session code depend on it instead of Network.framework.

The command domain includes arming, drive, stop, emergency stop, reset, video messages, and operator text. `RobotCommandEnvelope` carries protocol version, session UUID, monotonically increasing sequence, issue time, lease duration, and command. These fields make freshness and session binding explicit.

Numeric value objects sanitize at construction. `MotionVector`, `CameraVector`, and `TorsoVector` convert non-finite values to zero and clamp finite values to -1 through 1. `ControllerPose` rejects non-finite data, negative timestamps, and implausible quaternion magnitude, then normalizes the quaternion.

This technique prevents invalid floating-point values from spreading, but it should not silently conceal every programming error. Add debug assertions or diagnostics at trusted call sites when sanitization occurs.

Codable enums use an explicit `type` discriminator and associated payload keys. This is more stable and reviewable than relying on synthesized representation for a protocol shared between repositories.

CHAPTER 8

Treat dead-man logic as a pure decision function

SOURCE TRAIL — ANALYZING NOW: `DeadManController.swift` and its matching tests in the package. The source map prints their full paths.

`DeadManController` is a value type. It remembers armed state, latched emergency stop, scene activity, latest sequenced sample, receipt time, and a forced inhibit reason. Its default input timeout is 250 milliseconds.

Evaluation follows a deliberate precedence:

1. emergency stop;
2. connection readiness;
3. active scene;
4. armed state;
5. forced inhibit;
6. presence of input;
7. held dead-man state;
8. input age;
9. drive decision.

That ordering makes failures deterministic. A diagnostic controller pose may survive into a stop decision, but motion, camera, gripper, and torso authority do not.

Arming does not manufacture a fresh input. Resetting emergency stop disarms. A new sample must have a greater sequence than the retained sample. Use `ContinuousClock`, not wall-clock time, for local lease age because wall time can jump.

Tests should cover equality at the deadline, stale sequence, NaN clamping, disconnect, scene transitions, arming while E-stop is latched, reset behavior, and release.

CHAPTER 9

Translate controller hardware into one sample

SOURCE TRAIL — ANALYZING NOW: `Platform/GameController/GameControllerInput.swift` and `Resources/Info.plist`.

The app declares spatial and extended gamepad profiles. `GameControllerInput` observes connect/disconnect notifications, configures existing devices, starts wireless discovery, and stores state by object identity.

A conventional extended gamepad controls both treads. The left and right thumbstick Y axes remain independent tank-drive demands. The right thumbstick also supplies camera axes for compatibility. A conventional dead-man is A **or both shoulder buttons**. Index triggers remain reserved for grippers.

PSVR Sense-style spatial controllers arrive as `GCPhysicalInputProfile` rather than `GCExtendedGamepad`. The first supported one is assigned left, the next right. Singular and left/right element names are probed because SDK and controller mappings differ. Each controller's grip is the hold gesture; its index trigger becomes that side's Boolean gripper request.

Two VR grips must be held in the combined sample. Releasing either removes the continuous dead-man. Trigger values use a threshold above 0.5. Gripper commands are desired open/closed states, not measured jaw position.

Callbacks are not trusted as an everlasting stream. The implementation also polls `lastEventTimestamp` every 50 milliseconds and processes only a strictly newer timestamp. Missing fresh callbacks must lead to expiry in the session rather than replaying retained stick values.

Combine two controllers carefully

Per-device state contains side, last timestamp, primary stick, second stick, camera values, optional pose, grip, and trigger. Combination must be independent of dictionary iteration order. Left and right sides populate their corresponding tread, trigger, and pose. The aggregate `deadManIsHeld` requires the intended hardware combination.

When controller mappings change, add fixture logging on a development build, update the lookup table, and keep unknown element names out of production logs if they reveal device details.

CHAPTER 10

Convert tank drive without losing information

SOURCE TRAIL — ANALYZING NOW: `RobotViewModel.sendCombinedControllerSample()`.

The wire domain uses linear and angular motion while VR sticks produce left and right tread values. The exact reversible conversion used by the app is:

```
let linear = (leftTread + rightTread) * 0.5
let angular = (leftTread - rightTread) * 0.5
```

The receiver can reconstruct left as `linear + angular` and right as `linear - angular`, subject to later limiting. Multiplying the two resulting components by one speed limit preserves their relationship.

Every combined sample increments a wrapping `UInt64` sequence, declares `.gameController`, includes head-derived camera and torso values, carries gripper Booleans and optional controller poses, and repeats the dead-man state. The session—not the input class—decides whether the sample has authority.

CHAPTER 11

Track head orientation relative to consent

SOURCE TRAIL – ANALYZING NOW: `Platform/HeadOrientationInput.swift`.

`HeadOrientationInput` owns an `ARKitSession` and `WorldTrackingProvider`. It polls a device anchor every 40 milliseconds using `CACurrentMediaTime`. It does nothing unless the dead-man is held.

The first tracked orientation after the hold begins becomes the baseline. Relative orientation is:

```
let relative = baseline.inverse * orientation
let forward = simd_normalize(relative.act(SIMD3<Float>(0, 0, -1)))
let yaw = atan2(-forward.x, -forward.z)
let pitch = asin(max(-1, min(1, forward.y)))
```

Yaw maps to full camera pan at ±60 degrees. Pitch maps to full camera tilt at ±35 degrees. Both normalize to -1 through 1. Torso rotation remains zero inside the neck range; excess yaw between 60 and 180 degrees maps to normalized torso demand. Cerebro owns the Pololu Tic safety and physical position conversion.

Releasing the dead-man clears the baseline and emits unavailable. Tracking loss while held also emits unavailable. This prevents an old head pose from remaining active. Re-engaging naturally recenters at the current viewing direction.

Coordinate-frame bugs are common here. Record whether a transform maps anchor to origin or origin to anchor, define forward, and test signed yaw and pitch with known poses. Never “fix” one direction by adding unexplained negations downstream.

CHAPTER 12

Understand controller pose as telemetry, not authority

SOURCE TRAIL — ANALYZING NOW: `GameControllerInput.swift`, `ControlProtocol.swift`, and `TelemetryPanel.swift`.

Spatial controller poses are normalized, timestamped data included in a control sample and reflected in diagnostics. They do not bypass arming, dead-man, freshness, collision policy, or the robot's physical stop.

The position and quaternion describe a pose in the framework's tracking frame. They are not automatically an AMBER arm joint solution. Turning pose into arm motion requires calibrated transforms, workspace constraints, inverse kinematics, self/robot/environment collision checks, rate limiting, joint feedback, and cancellation behavior.

Keep raw pose, calibrated target, planned joint state, commanded joint state, and measured joint state as separate types. A single seven-float array cannot express their different truth claims.

CHAPTER 13

Jog two AMBER arms without pretending controller pose is inverse kinematics

SOURCE TRAIL — ANALYZING NOW: the arm protocol, dual-arm jog mapper, robot session, arm-control panel, and matching tests. The source map gives each exact path.

The current `rob-arm-control/2` path gives each arm its own measured feedback, authority UUID, lease, selected joint, draft, in-flight target, disposition, and hold state. Drive and direct arm control remain mutually exclusive, but left and right arm authorities are independent: either arm can be granted alone, and both can operate concurrently after separate preflight.

The Vision app never activates AMBER hardware or changes actuator modes. Before one arm may move, the transport must advertise physical arm execution, measured telemetry must be no more than 250 ms old, all seven joints must report position mode, the measured baseline must satisfy B1 bounds, and Cerebro must grant that exact paired controller and live session a time-limited authority. Targets carry identity, session, arm, increasing sequence, authority, issue time, short lease, dead-man state, seven bounded joint positions, and duration.

On-screen lanes allow one selected joint per arm to move by at most 0.08 radian per segment and an average requested rate no faster than 0.20 radian per second. Only one target per arm may be in flight. Releasing a lane requests a measured-position hold for that arm without silently moving the other.

With two paired PSVR Sense controllers and both authorities active, the left and right vertical thumbsticks jog the matching selected joints simultaneously. The mapper applies a 0.15 dead zone, copies the other six joints from fresh measured state, and preserves the same increment/rate/lease gates. Both grips form one paired dead-man: releasing either grip cancels both loops and asks both arms to hold while retaining their independent authorities. A fresh two-grip hold is required to resume.

Controller disconnect, more than 250 ms of input silence, stale telemetry, mode/preflight loss, target timeout/send failure, scene loss, stop, drive disarm, or software E-stop takes the stronger priority path: request both arms hold and clear local arm latches. Cerebro and the AMBER gateway still own per-arm identity, watchdog, measured completion, lease expiry, and hold-on-failure. The simulator advertises no physical arm execution, and hardware-in-the-loop validation remains required.

This is bounded measured joint jogging, not controller-pose IK. The tracked pose described in the previous chapter remains diagnostic. Cartesian teleoperation would require calibrated transforms, inverse kinematics, workspace/collision constraints, rate and force policy, singularity handling, and verified feedback that this path does not implement.

CHAPTER 14

Design press-and-hold spatial controls

SOURCE TRAIL — ANALYZING NOW: `ControlPanel.swift` and the `beginVirtualMotion` method in `RobotViewModel.swift`.

The UI alternative to a gamepad uses press-and-hold controls. Beginning a hold checks active scene, ready connection, armed state, and unlatched emergency stop. It captures speed-limited linear/angular values and starts an 80-millisecond refresh task. Ending cancels the task and explicitly invalidates input as dead-man released.

Do not implement motion as a one-shot button tap followed by a timer on the robot. The client should continuously renew a short lease, and the receiver should independently stop when renewal ceases.

SwiftUI gestures can end through release, cancellation, focus change, view removal, scene inactivity, or application termination. Route every observable end into the same invalidation method. Still assume the client can disappear without calling it; the robot-side watchdog is mandatory.

CHAPTER 15

Make controller ownership visible and transferable

The most recent authenticated operator to choose **Request Control** becomes the authoritative drive owner. Cerebro first stops the previous owner's motion, then broadcasts the new owner. **Release Control** brakes ROB and returns drive ownership to Cerebro. This is explicit arbitration, not a race between whichever client packet arrived last.

Each iPhone/iPad and Vision Pro needs its own pairing credential. Authentication establishes identity; the ownership state establishes which authenticated operator may currently send drive demands. A camera subscription, arm authority, face recognition result, or AI approval cannot substitute for drive ownership. Disconnection, revocation, release, or takeover invalidates the prior owner's active motion path.

The UI must display the authoritative server-reported owner rather than inferring ownership from a local button toggle. Test simultaneous requests, takeover during nonzero demand, old-client replay, disconnect, reconnect, and revocation. The expected transition includes a brake before the new owner can renew motion.

CHAPTER 16

Pair identity, not merely an address

SOURCE TRAIL — ANALYZING NOW: the credential, pairing-store, pinned-TLS, control-discovery, and pairing-sheet files. Exact names and paths are in the source map.

The user pastes a complete `ROBCTL2:` enrollment code. Installation parses and stores a unique controller credential plus Cerebro certificate pin in Keychain. "Credential installed" means local storage succeeded; only a live pinned-TLS and reciprocal-proof exchange earns "Connected and verified."

Bonjour `_robctl._udp` discovery filters for the paired robot identity. Each physical controller needs a distinct credential so it can be revoked and duplicate sessions can be detected. Never copy another controller's Keychain item.

Pinned TLS prevents a same-LAN service with another certificate from silently impersonating Cerebro. Reciprocal HMAC proof binds both peers to the pairing secret and role. The accepted handshake establishes the exact live session UUID used in subsequent envelopes and video authorization.

Certificate rotation must be intentional. An unexpected pin change should fail closed; deleting pin checks to make a demo connect converts an availability problem into an authentication vulnerability.

CHAPTER 17

Separate control and video failure domains

SOURCE TRAIL — ANALYZING NOW: `CerebroRobotTransport.swift`, the control client, the video client, and both discovery files.

Control uses `_robctl._udp` with application protocol `robctl/2`. Video uses `_robvideo._udp` with `robvideo/1`. Both are QUIC/TLS services, but they have distinct discovery, authentication exchanges, clients, framing, and lifecycle.

The authenticated control connection creates the live session UUID. Video proof and subscription carry that exact UUID, and the client opens the ordered QUIC media stream only after authentication. Disconnecting or replacing control, revoking credentials, backgrounding, or unsubscribing tears down associated video. An immersive scene transition may preserve media while independently braking drive. Video discovery or decode failure must not disconnect authenticated motion control.

This separation prevents megabytes of media and decoder backpressure from delaying a stop command. It also keeps optional camera availability from becoming a prerequisite for control.

CHAPTER 18

Negotiate video before allocating a decoder

SOURCE TRAIL — ANALYZING NOW: `Video/VideoProtocol.swift`, `ROBVideoWireTypes.swift`, and `RobotViewModel.videoRequest`.

After authentication, Cerebro sends bounded camera capabilities. The client may independently request `front`, `belly`, and `insta360` camera IDs with preferred codecs, dimensions, frame rate, bitrate, and delivery mode. Front and belly request H.264 at most 960 by 540; Insta360 requests 960 by 480. Every feed is capped at 20 frames per second and 1.5 Mbit/s. Production selects `reliableStream`; the synthetic simulator uses its in-memory datagram-shaped path.

The wire mapper rejects empty or oversized identifiers, duplicate cameras, zero dimensions, limits above hard caps, missing H.264, missing reliable-stream support, unknown JSON keys, and unsupported delivery combinations.

Only an accepted response creates a `VideoStreamDescriptor`. The descriptor carries session, subscription, camera identity, codec, geometry, rate, bitrate, and delivery facts used to validate subsequent media. Each camera owns a distinct pipeline coordinator and lifecycle task, so three accepted subscriptions can decode concurrently without one feed replacing another.

Subscription code uses a unique ID and a continuation. The session rejects duplicate IDs, times out the request, handles cancellation, and remembers abandoned IDs so a late response cannot create an unwanted stream.

CHAPTER 19

Frame ordered QUIC video defensively

SOURCE TRAIL — ANALYZING NOW: `Video/ROBVideoFramer.swift`, `ROBVideoWireTypes.swift`, and `ROBVideoAuthentication.swift`.

The ordered stream uses a fixed 32-byte big-endian `RVID` header:

Offset	Size	Field
0	4	magic `RVID`
4	1	protocol version
5	1	header size
6	2	message type
8	4	payload length
12	4	reserved zeros
16	8	increasing sequence
24	8	reserved zeros

The framer rejects invalid magic, version, size, type, reserved bits, excessive payload, and non-increasing input sequence. Control JSON is capped at 64 KiB. Codec configuration is capped at 64 KiB. Access units are capped at 2 MiB, and total framed media has a derived maximum.

The Network framework framer separates message boundaries from stream reads. Its parser waits for an exact header; no-copy delivery emits one validated payload with metadata. Output writes the header and payload without copying when possible.

Sequence protects stream logic from duplicate or reordered application frames; TLS provides confidentiality and integrity. It is not a replacement for the session/stream/access-unit sequence validation inside the media domain.

CHAPTER 20

Carry codec configuration and access units explicitly

SOURCE TRAIL – ANALYZING NOW: `EncodedVideoProtocol.swift`, the H.264 sample-buffer factory, and the H.264 receiver.

H.264 decoding needs parameter sets and access units. The protocol sends codec configuration separately with SPS/PPS bytes, NAL length-field size, generation, session ID, and stream ID. Access units carry sequence, presentation timestamp, timescale, duration, keyframe flag, and AVCC payload.

`VideoStreamValidator` binds every message to the negotiated session and stream, checks configuration generation, sequence, timestamps, dimensions, and keyframe recovery state. A gap makes predictive frames unsafe because they may depend on a missing reference. The receiver drops them, requires a new keyframe, and sends bounded feedback.

The H.264 sample-buffer factory creates a Core Media video-format description from parameter sets and constructs timed sample buffers from AVCC payload. It validates expected dimensions rather than trusting the bitstream to allocate arbitrary surfaces.

Never assume Annex B start codes and AVCC length-prefixed NAL units are interchangeable. Convert at one named boundary and test both malformed and valid fixtures.

The first H.264 NAL byte contains a forbidden-zero bit, two reference-priority bits, and a five-bit type. The current profile cares especially about type 1 non-IDR slices, type 5 IDR slices, type 7 SPS, and type 8 PPS. SPS/PPS travel in the configuration message. An AVCC access unit is a repetition of `[big-endian length][NAL bytes]`, where the negotiated length field is 1, 2, or 4 bytes. A key-frame flag must agree with a type-5 NAL; after a sequence gap the receiver discards predictive type-1 data until fresh configuration and an IDR restore a known reference state.

This application does not use RFC 6184 RTP packetization. There are no STAP-A aggregation packets or FU-A fragments in the current wire format. Complete bounded AVCC access units ride inside the 92-byte `RBVD` media header, which rides inside the 32-byte ordered `RVID` QUIC frame. Keep those three layers distinct when diagnosing "bad H.264": stream framing can fail before media framing, and media framing can succeed while the decoder rejects SPS/PPS or slices.

CHAPTER 21

Decode without blocking control or the main actor

SOURCE TRAIL — ANALYZING NOW: the H.264 receiver in the video library and the pipeline coordinator in the application. Their exact filenames are in the source map.

`H264VideoReceiver` is an actor. It consumes an `AsyncThrowingStream`, updates statistics, validates messages, configures the sample-buffer factory, and enqueues accepted frames. Decoder pressure never waits indefinitely: renderer readiness gets a 50-millisecond bounded wait with two-millisecond cancellation-aware polling.

When the renderer fails or requires a flush, the receiver drops the access unit, flushes, requires a keyframe, and requests one. Keyframe requests are rate-limited to one per 500 milliseconds during continuing recovery. Statistics publish at most twice per second unless forced.

`AVSampleBufferVideoRenderer` is documented for background enqueueing, but its SDK annotation does not satisfy Swift 6 isolation when obtained from a main-actor display layer. The code uses a narrow `@unchecked Sendable` handle that records this framework guarantee. Do not spread `@unchecked Sendable` across the display layer or arbitrary UIKit objects; isolate the exception and explain it.

`VideoPipelineCoordinator` remains `@MainActor` because it owns display state. A lifecycle generation and pipeline UUID prevent an old open/close operation from taking over a replacement pipeline. It closes an unowned channel on every failed handoff, avoiding media-channel leaks.

CHAPTER 22

Present three AVFoundation feeds inside SwiftUI

SOURCE TRAIL – ANALYZING NOW: `SampleBufferVideoView.swift` and `VideoPanel.swift`.

`SampleBufferVideoView` is a `UIViewRepresentable`. Its host view owns no decoder; it attaches the coordinator's `AVSampleBufferDisplayLayer`, removes any prior layer, uses `.resizeAspect`, and updates the layer frame in `layoutSubviews` with implicit animations disabled.

`dismantleUIView` detaches the display layer. This matters when SwiftUI reconstructs or removes the representable. A display layer must not remain attached to two superlayers.

`VideoPanel` treats availability, subscription, pipeline state, and statistics as different concepts for each camera. A connected control session may have no cameras. One camera may exist while another is absent. A subscription may be accepted while only its decoder is starting or failed. The front panel remains the largest fast-control view; belly and panoramic feeds have independent enable controls.

Keep the camera large and visually primary, but do not overlay controls that can hide connection or safety state.

Put the panorama inside a mixed-immersive sphere

`insta360` can remain in a flat two-to-one window or feed `Insta360ImmersiveView`. The immersive renderer creates a large sphere, culls its outward-facing triangles so the interior is visible, converts decoded pixel buffers into replaceable RealityKit textures, and maps the live panorama onto that interior. A blue diagnostic surface appears when valid texture data is unavailable; a black sphere is not accepted as "probably loading."

The current hardware alignment starts at -90 degrees. Left/right step controls, a continuous heading slider, and **Face Robot Front** let the operator correct/reset view heading without changing robot pose. Orientation signs must be verified on the physical headset because a plausible simulator panorama can still be mirrored or rotated.

The immersive space uses `.mixed` because the full compositor path produced a black live texture on the tested Vision Pro. Main fast-control windows remain visible. During a window-to-immersive transition the app preserves the authenticated session, subscription, and decode pipeline but invalidates motion. Only backgrounding performs full media teardown. This is a deliberate separation between "keep seeing" and "keep driving."

CHAPTER 23

Understand audio: capture locally, transmit text

SOURCE TRAIL — ANALYZING NOW: `VisionSpeechInput.swift`, `OperatorSpeechPanel.swift`, the view model's text-send method, and `ControlProtocol.swift`.

The current Vision Pro controller does **not** stream microphone audio to Cerebro. Its audio path is:

```
Vision Pro microphone
  -> AVAudioEngine input node
  -> 1024-frame tap
  -> SFSpeechAudioBufferRecognitionRequest
  -> SFSpeechRecognizer
  -> partial/final transcript
  -> editable SwiftUI String
  -> OperatorTextMessage over authenticated control
```

The input first requests Speech authorization, then microphone permission. It creates a recognizer for the current locale, verifies availability, enables partial results, and requires on-device recognition when the device supports it:

```
request.shouldReportPartialResults = true
request.requiresOnDeviceRecognition = recognizer.supportsOnDeviceRecognition
```

That assignment does not promise every locale or OS state can recognize offline; it requests on-device operation when supported. The UI must report unavailability rather than silently changing privacy behavior.

The `AVAudioEngine` input node installs a bus-zero tap with a 1024-frame buffer and the node's output format. Each buffer is appended to the recognition request. `prepare()` and `start()` may throw. Every failure and stop path removes the tap, ends/cancels recognition, clears retained objects, and updates UI state.

Recognition callbacks enter a `Task { @MainActor ... }` before touching observable state. Partial transcripts update the draft. A final transcript invokes the callback with `isFinal`, updates status, and stops capture.

Command versus puppet speech

`OperatorTextMode.command` asks Cerebro to process text like its local text input. `puppetSpeech` asks ROB to speak the text verbatim without sending it to the AI or motion parser. Both modes carry a Codable `OperatorTextMessage`; neither arms motion.

The current initializer automatically sends a final dictation using the mode selected at completion time. The panel also offers explicit Send as Input and ROB Says It buttons. For higher-consequence installations, consider changing auto-send to review-before-send, especially when ambient speech may be misrecognized.

If true audio streaming is added later

Treat it as a new protocol, not an extra field in operator text. Specify format, sample rate, channel count, frames per packet, timestamps, clock domain, jitter buffer, congestion/backpressure policy, echo cancellation, interruption handling, authentication, consent indication, retention, and receiver ownership. Keep it off the motion channel. Add Info.plist disclosure and an unmistakable live microphone indicator.

CHAPTER 24

Declare privacy and device capabilities

SOURCE TRAIL — ANALYZING NOW: `Resources/Info.plist`.

The plist declares:

- local-network usage for robot discovery;
- Bonjour service types `_robctl._udp` and `_robvideo._udp`;
- microphone use for explicit dictation;
- speech recognition for conversion to reviewable text;
- accessory tracking for spatial-controller pose;
- spatial and extended game-controller profiles;
- the main cockpit window, an Insta360 window, and a mixed Insta360 immersive space.

Usage descriptions should say what data is used for and when. They are user communication, not mere App Store obstacles. Adding raw audio transmission would make the present wording incomplete.

Test permission states independently: not determined, denied, restricted, authorized, recognizer unavailable, microphone interruption, local network denied, and accessory tracking unavailable.

CHAPTER 25

Design failures as ordinary states

SOURCE TRAIL — ANALYZING NOW: the connection-status view, telemetry panel, Cerebro transport errors, and video pipeline errors.

The UI should distinguish disconnected, connecting, handshaking, connected, disconnecting, and failed. Installed pairing is not verified pairing. Video unavailable is not control unavailable. Speech unavailable is not motion failure. Tracking unavailable must neutralize camera/torso demand without inventing a pose.

Prefer typed error cases with `LocalizedError` descriptions at presentation boundaries. Avoid logging secrets, pairing codes, HMAC material, certificate private keys, or raw dictated content by default.

Status strings help a human, but state must remain typed. Do not make button enablement depend on parsing a status message.

CHAPTER 26

Test the package in layers

SOURCE TRAIL – ANALYZING NOW: all files under `Packages/ROBControlCore/Tests`.

The package includes:

- dead-man transition and timeout tests;
- simulator connection and watchdog tests;
- video-domain Codable and validation tests;
- synthetic pixel-buffer and end-to-end pipeline tests;
- legacy payload compatibility tests;
- control wire, pairing, pinning, and authentication tests;
- video wire compatibility tests.
- independent arm-authority, target, hold, stale-feedback, and dual-jog mapping tests;
- multi-camera subscription and immersive-launch smoke hooks.

Write pure tests for value normalization and state machines first. Use actors and asynchronous streams in integration tests. Give every async expectation a deadline. Explicitly cancel tasks and close streams so a passing test does not leak work into the next test.

The simulator is a protocol peer, not a visual mock. It independently evaluates input age, integrates a deterministic pose, reports telemetry, negotiates video, and serves synthetic H.264 through the same domain boundaries.

Package tests cannot validate visionOS focus delivery, actual controllers, ARKit tracking, Local Network permission, Bonjour on a LAN, Keychain entitlements, camera capture on Cerebro, decoder behavior on device, or physical stopping. Maintain a separate two-device checklist.

CHAPTER 27

Debug without drowning Xcode

> **SOURCE TRAIL — ANALYZING NOW:** `README.md` diagnostics plus the relevant transport state handlers.

Use focused evidence:

```
dns-sd -B _robctl._udp local.
dns-sd -B _robvideo._udp local.
```

Then observe one layer at a time: controller connection and timestamp, authoritative owner, app focus, dead-man sample, session decision, per-arm authority/feedback, control handshake, command acknowledgement, video authentication, camera ID/subscription, codec configuration, access-unit sequence, flat/immersive render statistics, and heading.

Rate-limit repetitive logs. The earlier "bad file descriptor" style flood can make the debugger unusable and hide the first failure. Log state transitions and counters, not every poll. Use unified logging categories with privacy annotations in a future hardening pass.

CHAPTER 28

Add a feature without breaking authority

SOURCE TRAIL — ANALYZING NOW: use the whole chain for the nearest existing feature before editing.

For a new controller gesture:

1. identify the physical element and supported profiles;
2. add per-device state without changing dead-man semantics;
3. combine deterministically;
4. add a bounded domain type;
5. include it in the control sample and decision;
6. encode it in an explicit protocol field;
7. update the matching Cerebro decoder;
8. keep execution behind robot-side state and limits;
9. add fixture and state-machine tests;
10. test simulator, then two devices, then restrained hardware.

For a new video codec, update capability negotiation, hard limits, wire representation, configuration records, decoder factory, recovery rules, fixtures, and robot encoder. Do not select a codec merely because both platforms expose a framework enum.

For a new speech mode, add a Codable enum case, deterministic Cerebro routing, UI explanation, backward-compatibility behavior, and tests proving it cannot arm or create raw motion.

CHAPTER 29

Swift techniques worth carrying elsewhere

The codebase demonstrates several reusable techniques:

- `@MainActor @Observable final class` for presentation coordination;
- `@ObservationIgnored` for internal tasks and services;
- actors for mutable session and media state;
- immutable Sendable value objects at boundaries;
- explicit clamping and finite-number validation;
- dependency inversion through transport protocols;
- AsyncStream for state/event delivery;
- checked continuations with cancellation and deadlines;
- generation tokens against stale async completion;
- weak callback capture against ownership cycles;
- `ContinuousClock` for leases and local timeouts;
- narrow, documented `@unchecked Sendable` framework adapters;
- UIViewRepresentable for a carefully owned AVFoundation surface;
- separate optional media and authoritative control failure domains;
- explicit protocol versions, discriminators, limits, reserved fields, and sequence numbers.

None of these techniques is automatically correct. Their value comes from matching ownership and failure behavior to the real system.

CHAPTER 30

Production review checklist

- ☐ package tests pass under the committed Swift toolchain;
- ☐ visionOS simulator target builds without signing;
- ☐ bundle ID, team, and entitlements are intentionally configured;
- ☐ every device has a unique pairing credential;
- ☐ certificate pin and reciprocal proof fail closed;
- ☐ app focus and scene inactivity invalidate input;
- ☐ 250 ms client lease and Cerebro watchdog are measured together;
- ☐ controller disconnect and missing callbacks stop motion;
- ☐ both VR grips implement the intended dead-man gesture;
- ☐ index triggers affect only the matching gripper request;
- ☐ Request/Release Control displays the server-authoritative owner and brakes before takeover;
- ☐ drive and direct arm control remain mutually exclusive;
- ☐ each arm requires its own fresh telemetry, position-mode preflight, authority UUID, lease, measured baseline, bounded target, and hold result;
- ☐ paired arm jogging enforces both grips, 0.15 dead zone, 0.08-radian segments, 0.20-radian/second rate, one in-flight target per arm, and priority hold on every stale/fault path;
- ☐ tracked controller pose remains diagnostic and cannot enter the joint-target protocol;
- ☐ head baseline resets on every hold and tracking loss neutralizes output;
- ☐ torso rotation begins only outside the neck range;
- ☐ video failure never delays or disconnects control;
- ☐ video messages are session/stream/sequence bounded;
- ☐ front, belly, and Insta360 subscriptions use distinct authenticated pipelines and the intended 960-by-540 or 960-by-480 ceilings;
- ☐ decoder pressure drops or recovers media rather than blocking;
- ☐ immersive transitions brake motion while preserving only the intended live media; backgrounding tears it down;
- ☐ the -90-degree panoramic start, step/slider controls, and Face Robot Front reset are verified on the physical headset;
- ☐ microphone and speech permissions are understandable;
- ☐ documentation states that microphone audio is local and only text is transmitted;
- ☐ command and puppet-speech routes remain distinct;
- ☐ logs exclude secrets and dictated content by default;
- ☐ simulator, two-device, and restrained-hardware tests are recorded;
- ☐ physical emergency stop behavior is independently verified.

CHAPTER 31

File-by-file reading order

Use this order for a new Swift contributor:

1. `README.md`
2. `Packages/ROBControlCore/Package.swift`
3. `Control/ControlProtocol.swift`
4. `Control/DeadManController.swift`
5. `Control/ArmControlProtocol.swift` and `DualArmJointJogMapper.swift`
6. `Connection/RobotSession.swift`
7. `Simulation/SimulatedRobotEndpoint.swift`
8. `App/RobotViewModel.swift`
9. `App/ROBControllerVisionApp.swift` and `App/ContentView.swift`
10. `Platform/GameController/GameControllerInput.swift`
11. `Platform/HeadOrientationInput.swift`
12. `Platform/VisionSpeechInput.swift`
13. `Features/Control/ControlPanel.swift` and `ArmControlPanel.swift`
14. `ROBCerebroTransport/CerebroRobotTransport.swift`
15. control discovery, ownership, wire, client, and pairing files
16. `Video/VideoProtocol.swift` and `EncodedVideoProtocol.swift`
17. video discovery, authentication, framer, wire types, and client
18. `H264VideoReceiver.swift` and `H264SampleBufferFactory.swift`
19. the front, belly, and Insta360 `VideoPipelineCoordinator` ownership in `RobotViewModel`
20. `SampleBufferVideoView.swift`, `VideoPanel.swift`, and the immersive renderer
21. every corresponding test before making a change

When a shortened path is ambiguous, use the public source-code map[1] or search inside the cloned repository:

```
rg --files ROBControllerVision | rg 'DeadManController|ArmControl|VisionSpeechInput|ROBVideoFramer|VideoPanel'
```

[1] https://github.com/RudyAramayo/ROBBooks/blob/main/ROB-Books/OPEN-SOURCE-CODE-MAP.md

CHAPTER 32

Closing principle

ROBControllerVision is compelling because Vision Pro can combine a large first-person camera, spatial controls, tracked head orientation, controller pose, dictation, and a wraparound instrument deck. It is trustworthy only when those rich inputs are translated into small, typed, bounded, expiring requests.

The most important Swift skill in this controller is not a particular modifier or framework call. It is making ownership visible: which actor owns state, which task owns work, which session owns a message, which view owns a layer, which controller owns an input, which permission owns capture, which receiver owns a timeout, and which physical mechanism remains outside the app's authority.